



Tecnológico
de Monterrey

05. Vibe-coding

Ciencia de Datos II

Asistentes de código: el humano dirige, la IA teclea
24 de agosto de 2026

**Jorge
Juvenal
Campos Ferreira**

✉ juvenal.campos@tec.mx

Vibe Coding



El ★**vibe coding** es programar **describiendo la intención en lenguaje natural y dejar que un agente de IA escriba el código.**

El humano conduce, evalúa y corrige; no teclea syntaxis.

El nombre captura la esencia: estás "*vibrando*" con la máquina, comunicándole tu visión general y dejándola manejar la programación.

Es como ser el director de una película: tú defines la visión y la IA (el equipo de producción) ejecuta los detalles técnicos.

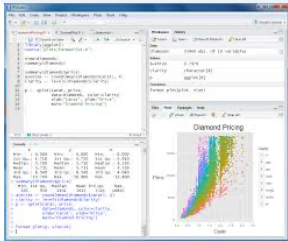


¿Que es vibe coding?

Antes

1. Aprender la sintaxis de R
2. Aprender tidyverse, ggplot2, sf...
3. Depurar hasta que corra
4. Producir un grafico

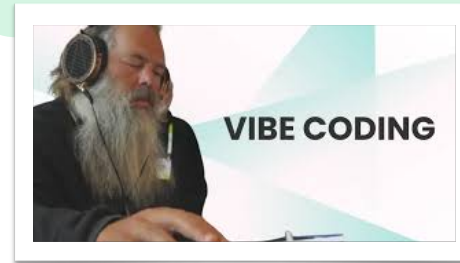
→ *semanas o meses de capacitación*



Con vibe coding

1. Describir el problema en español
2. Dar insumos (CSV, fuentes, formatos)
3. Leer y verificar el código que devuelve la IA
4. Iterar con feedback preciso

→ *minutos para un primer resultado*

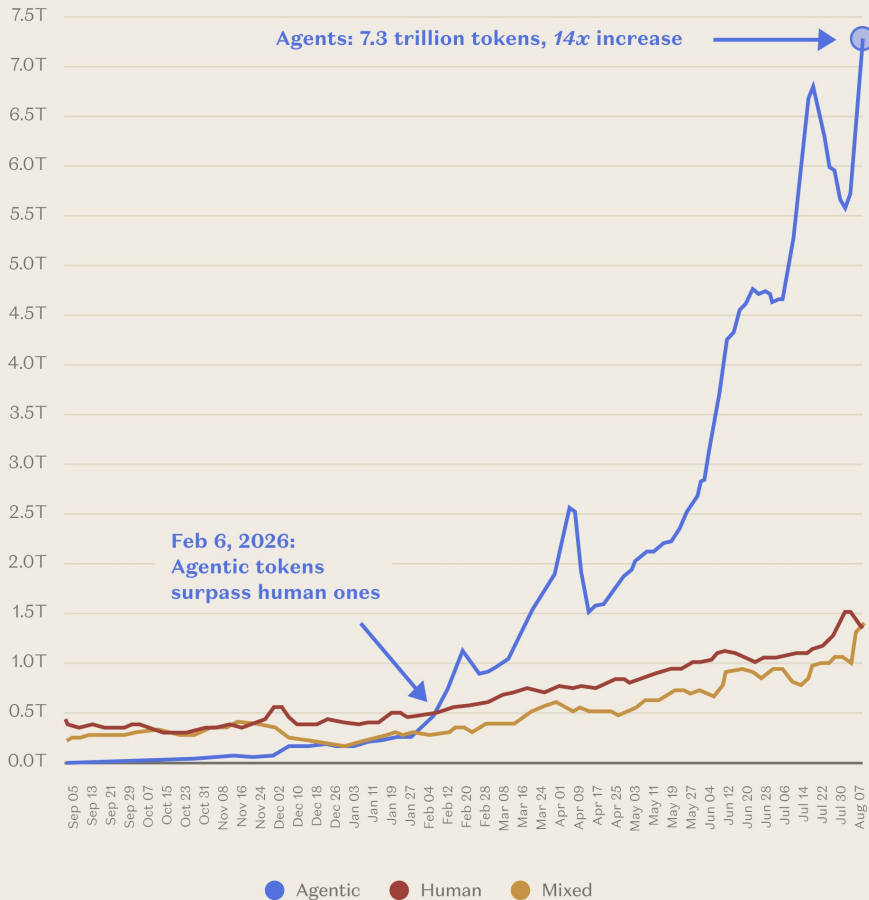


(!!!) El Vibe coding **NO elimina aprender a programar.**

Los agentes son los que usan los *tokens*

Agents Are Using Far More Tokens Than People

7-day average token usage on OpenRouter, by type



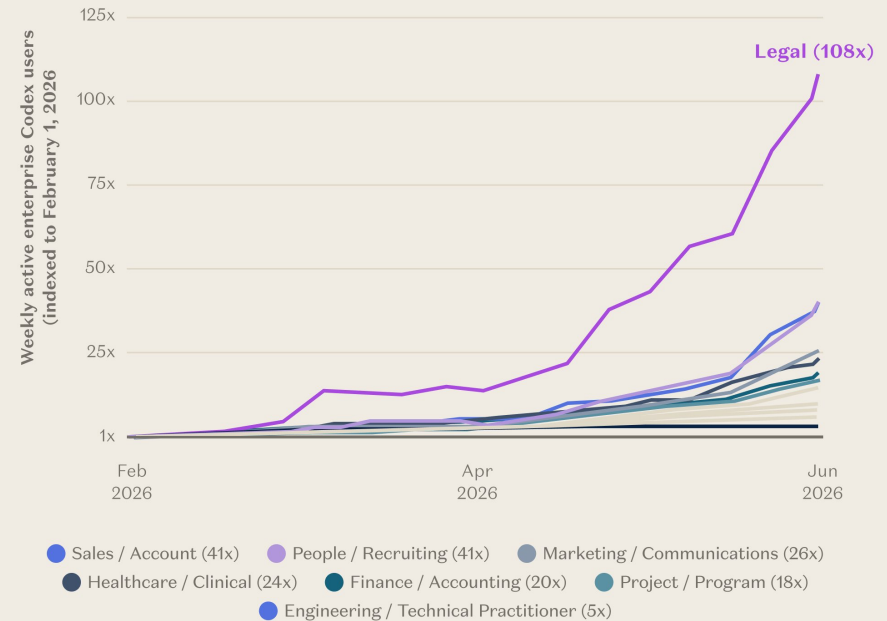
Source: openrouter.ai/rankings

More charts: a16z.news/subscribe

A16Z

Legal Is Driving Knowledge Work To The Frontier (Recently, At Least)

Codex user growth by enterprise job title



Source: OAI Signals, "What Frontier Firms are doing differently" Aug. 12, 2026.

More charts: a16z.news/subscribe

A16Z

La evidencia: acelera más los proyectos nuevos



+55.8% — Peng et al. (2023). Es el experimento clásico de GitHub Copilot. Reclutaron ~95 programadores freelance y les pidieron una tarea única y bien especificada: implementar un servidor HTTP en JavaScript desde cero, lo más rápido posible, con éxito medido por tests. **El grupo con Copilot terminó 55.8% más rápido.**

Tomar en cuenta:

1. **Tarea desde cero** (sin código previo).
2. **Escenario donde el modelo tenía todo el contexto necesario para hacer su tarea.**
3. **Los desarrolladores con menor experiencia se beneficiaron más.**

+26.1% — Cui et al. (2025). Tres experimentos de campo aleatorizados en Microsoft, Accenture y una empresa Fortune 100 anónima, con 4,867 desarrolladores en total, a quienes se les asignó acceso a Copilot al azar. La métrica principal fue el número de *pull requests* completados por semana, y el efecto agrupado fue ~26%.

Tomar en cuenta

1. **La métrica de éxito era la cantidad de outputs, no tiempo por tarea ni calidad**
2. **No se deterioró la calidad de los repositorios, por lo que no solo metieron basura**
3. **Los desarrolladores junior usaron más la herramienta que los senior**
4. **El trabajo corporativo mete más fricciones que el anterior experimento no tiene** (código legado, revisiones adicionales, mayores dependencias) **lo que podría comprimir el efecto del uso de *vibe coding*.**

-19% — METR (2025). RCT con 16 desarrolladores muy experimentados de proyectos open source grandes y maduros (repos de más de un millón de líneas, con años de trabajo), con 246 tareas reales asignadas al azar a "con IA permitida" (principalmente Cursor con modelos Claude de la época) o "sin IA".

Resultado: **con IA fueron 19% más lentos.** ¿Por qué? El experto ya tiene el contexto del repo en la cabeza — convenciones, historia, trampas conocidas — y su línea base es muy alta.

El modelo no tiene ese contexto tácito, así que propone código plausible pero desalineado e incompatible, y **el experto gasta más tiempo revisando, corrigiendo y re-prompteando de lo que le habría tomado escribirlo directo.**

Tomar en cuenta:

Cuando el contexto necesario no es transferible al modelo, el *vibe coding* puede estorbar.

Si la IA puede escribir código por mí, ¿Vale la pena entonces aprender a programar?

Si.

Antes aprender a programar era **la única forma** de crear software y hacer análisis de datos de forma eficiente y a escala. Aprender tomaba meses, dedicación y práctica.

Hoy también sirve para dirigir, evaluar y corregir lo que la IA produce. Sin fundamentos, las herramientas de IA son cajas negras por las que pasan cosas que no entendemos, y donde si algo sale mal, no sabemos por donde empezar a arreglarlo. Con **fundamentos**, estas mismas herramientas se vuelven **amplificadores**.

Además, aprender a programar permite:

- **Comunicar** con precisión los problemas que se quieren resolver
- **Disecionar** en partes, segmentos y módulos dichos problemas
- **Comprender** conceptos, herramientas y etiquetas técnicas: palabras y términos que te permiten pensar cosas que antes no podíamos pensar, pedir cosas que antes no podíamos pedir y entender y leer cosas que antes no podíamos entender.

Formas de trabajar con IA y código:

Chat

Promptear la necesidad,
copiar y pegar el código.

ChatGPT, Claude,
Gemini, chatbots con
interfaz web



Autocompletado

Sugerencia mientras
tecleas en el editor

Github copilot



Agente en editor

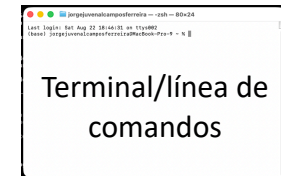
edita archivos con tu
permiso

Cursor



Agente CLI

lee, escribe y ejecuta en tu
computadora

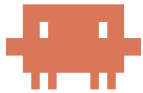


Más autonomía y capacidad de acción
Más responsabilidad de verificar el resultado

En este curso trabajaremos sobre todo con agentes CLI (interfaz de línea de comandos) que **leen y escriben archivos en tu carpeta, ejecutan el código y lo prueban.**



CLIs comerciales



Claude Code (Anthropic)

Requiere: Claude Pro o Max.

Fortaleza: código complejo y contexto largo; buen desempeño con R.



Codex (OpenAI)

Requiere: ChatGPT. Más uso en cuentas Plus, Pro o Business.

Fortaleza: integración fluida con el ecosistema de ChatGPT. Puede hacer imágenes en lote.



Gemini CLI (Google)

Requiere: Google AI Pro o llave de API.

Fortaleza: nivel (tier) gratuito generoso para talleres.



OpenCode

En usos básicos, puede ser más barato.

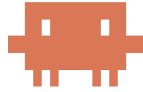
Fortaleza: utiliza niveles chinos de frontera y puede operar de interfaz con modelos locales.

Plan B: la interfaz web

Si la CLI no instala, claude.ai, gemini.google.com o chatgpt.com aceptan los mismos prompts. Se pierde la ejecución directa en tu disco; el código se copia a RStudio y se corre ahí.

Instalar una CLI desde la línea de comandos:

Claude Code



```
Mac (nativo):  
curl -fsSL https://  
claude.ai/install.sh | bash  
  
Mac (npm):  
npm install -g @anthropic-  
ai/claude-code  
  
Windows (PowerShell):  
irm https://claude.ai/  
install.ps1 | iex  
  
Windows (winget):  
winget install  
Anthropic.ClaudeCode
```

Gemini CLI



```
Mac (Homebrew):  
brew install gemini-cli  
  
Mac (npm):  
npm install -g @google/  
gemini-cli  
  
Windows (winget):  
winget install  
Google.GeminiCLI  
  
Windows (npm):  
npm install -g @google/  
gemini-cli
```

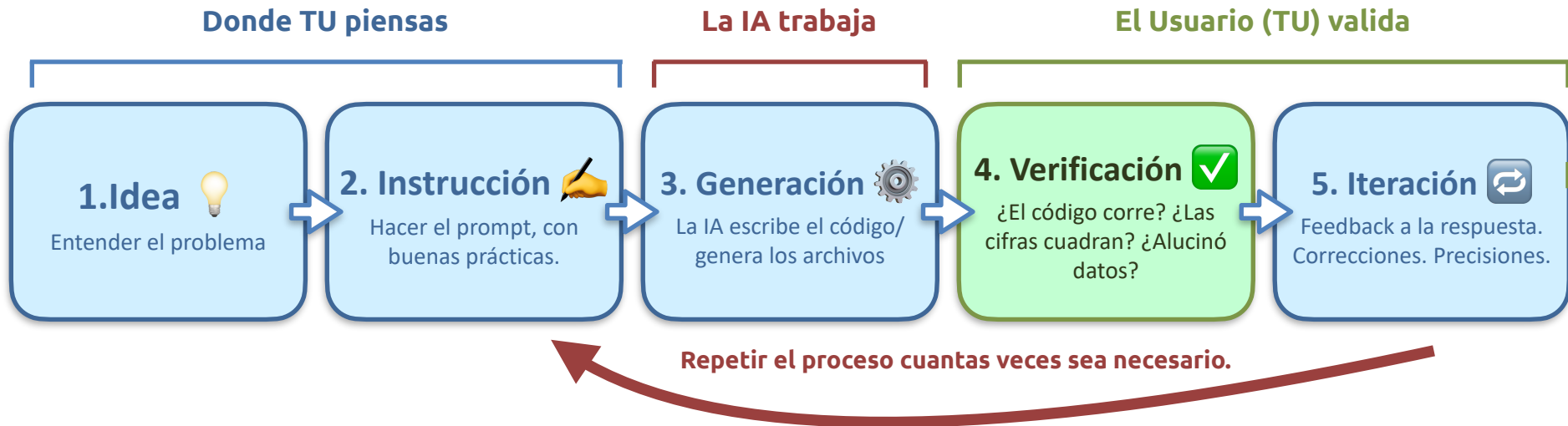
Codex



```
Mac (Homebrew):  
brew install --cask codex  
  
Mac (npm):  
npm install -g @openai/  
codex  
  
Windows (winget):  
winget install OpenAI.Codex  
  
Windows (npm):  
npm install -g @openai/  
codex
```

Verificar: `claude --version`, `gemini --version`, `codex --version`. Las npm requieren Node.js 18+ (nodejs.org).

Los pasos del vibecoding



Notas:

- 1) **La verificación es lo que separa al vibe-coding de un acto de fe.** Hay que tener forma de verificar que los datos que se generan/análisis que se hacen estén correctos.
- 2) **Cuidar el tamaño de la ventana de contexto:** llenarla consume más tokens y provoca más errores y alucinaciones
- 3) **Usar el modelo adecuado para la tarea adecuada.** Modelos más potentes pueden ayudar para la planeación; modelos más ligeros para las tareas sencillas.

Prompting efectivo en vibecoding

Elementos recomendados en un prompt de **vibecoding** (retomando de los vistos la semana pasada):

Tarea o meta

Resultado a esperar.
Meta a cumplir.
Trabajo a realizar



Contexto

Archivos de tu carpeta de trabajo a utilizar, memoria de la sesión, conversaciones previas, ejemplos importantes, información que le proporciones al agente, etc.



Restricciones

Uso de estándares, reglas, arquitectura, convenciones.
Definición de características deseables.



Ejemplos

Ejemplos del resultado o uso de **Anclas** o valores esperados en el resultado: "El saldo de 2024 debe rondar los 171 mil; si el resultado no coincide algo está mal, ya sea el dato, el código o tu supuesto."



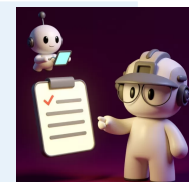
Criterio de éxito

Condición verificable que señala que terminó: test que se pasaron, *bugs* que ya no se reproducen, auditorías que se cumplen.



Modo Plan:

El modo planeación también es importante: activar ante tareas complejas/confusas/que puedan dar pie a ambigüedades.

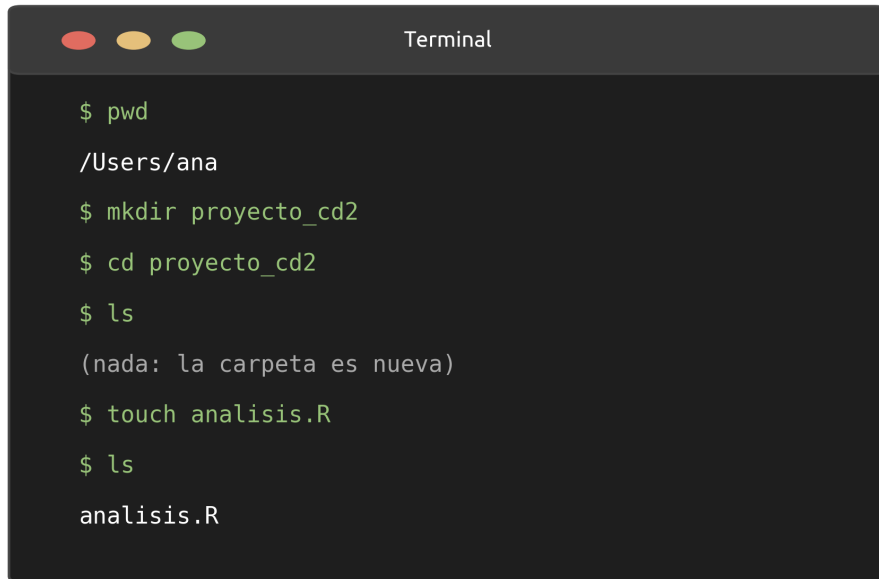


La terminal es la casa de los agentes

La terminal (línea de comandos) permite interactuar con el sistema operativo tecleando comandos línea por línea.

Los agentes CLI viven ahí: para dirigirlos hay que saber moverse en la terminal.

Una sesión típica: crear la carpeta de un proyecto



```
$ pwd
/Users/ana
$ mkdir proyecto_cd2
$ cd proyecto_cd2
$ ls
(nada: la carpeta es nueva)
$ touch analisis.R
$ ls
analisis.R
```

pwd: ¿dónde estoy parado?

mkdir: crea una carpeta

cd: entra a la carpeta

ls: lista los archivos

touch: crea un archivo vacío

*el sistema responde
con texto, no con ventanas*

cd es el comando más básico para el Vibe Coding: te permite moverte a tu carpeta de trabajo y lanzar el agente.

Cinco tipos de fallos del vibe coding

Alucinación de funciones

Llama funciones que NO existen (ggplot2::geom_choropleth()). Se ve razonable; no corre.

Código que corre... mal

Calcula importaciones menos exportaciones al revés. Sale gráfica, sin error, con el signo invertido.

Unidades equivocadas

Millones contra miles de millones; 'trillions' en inglés son billones en español. Un cero de más cambia la narrativa.

Atribuciones inventadas

Cita fuentes que no existen o que no dicen eso. Grave en textos que no se revisan.

Bucle de errores recursivos

El agente asegura que corrigió el error... sin corregirlo, una y otra vez. Señal de cortar y replantear el prompt.



Estos errores se van reduciendo conforme mejoran los modelos. Igualmente, a medida que estos mejoran, el cuello de botella se vuelve el usuario.



Cinco fuentes de fallos:

Contexto

¿El modelo tiene la información suficiente?

Capacidades

¿La IA sabía la respuesta y que hacer pero no podía hacer algo al respecto?

Reglas

¿Se hicieron las cosas en un orden que no era el correcto?

Memoria

¿La IA olvidó instrucciones que ya le habías dicho antes?

Validación

¿El agente tenía algún sistema que revisara su trabajo antes de devolverlo al usuario?

Ejercicio práctico

1. En una carpeta vacía, pidan a su **agente de programación** que construya, en un solo **archivo HTML** que se pueda abrir en el navegador, el juego del Snake. Verifique que se pueda jugar y que funcione correctamente. Guarde el resultado en la carpeta “ejercicio_01”.
2. En la carpeta “ejercicio_02” hay ofertas de empleo. Genere un *prompt* para que, tomando una sola oferta de empleo, el agente haga una búsqueda sobre la empresa, los posibles entrevistadores y los cursos y certificaciones que necesitaría tomar para cumplir el perfil. Itere y ajuste hasta que esté contento con el resultado. Una vez que esté bien, genere una **skill** o un **command** que pueda llamar para cuando le lleguen ofertas de trabajo nuevas.
3. En la carpeta “ejercicio_03” están los datos de las proyecciones de población estatales de CONAPO a 2070 y las proyecciones de población de CONAPO municipales a 2040. A partir de estos datos, y usando *vibe-coding* y R, realice lo siguiente:
 - a) Una gráfica de facetas que le permita ver la proyeccion poblacional de cada estado a 2070, además de conocer el año del pico poblacional. Guarde la imagen.
 - b) Una **aplicación shiny** que permita visualizar, seleccionando estado y municipio, la proyección de población a 2040 de un municipio seleccionado.
4. En la carpeta “ejercicio_04” están los insumos suficientes para replicar una aplicación que permite revisar, a nivel municipal, datos de preferencia electoral. El profesor proyectará la aplicación terminada. Por favor, utilizando **vibe-coding**, intente replicarla.

Funciones del Agente

/help: función que permite ver el catálogo de comandos en cada CLI

/model: permite ver y cambiar el modelo de IA que se está utilizando

@archivo_importante.R Arrobar archivos cuando nos queremos referir a ellos le permite al agente encontrarlos de forma más sencilla e inequívoca, además de cargar solo la información del archivo que se necesita.

/plan Activa el modo planeación

/permissions (Claude Code) - **/approvals** (Codex - Opencode) Para darle permisos al agente de hacer cosas sin verificarlo siempre contigo.

ESC (la tecla) - Para interrumpir al agente a media tarea. **Doble ESC** (Claude Code) para regresar a un mensaje anterior y re-escribirlo.

/rewind (Claude Code) y **/undo** (Codex) para revertir acciones hechas en un prompt

/context para revisar cuanta ventana de contexto ya tienen ocupada

/clear para borrar toda la ventana de contexto y comenzar una tarea nueva - **/compact** para resumir la ventana de contexto antes de que se desborde

/cost y **/usage** para ver cuanto va costando (en uso, en tokens o en dinero) lo que llevamos de la sesión

/init genera un archivo CLAUDE.md (Claude Code) o AGENTS.md (Codex) que funcionan como archivos de configuración - prompts del sistema del proyecto que voy a trabajar.

- Guarda una instrucción directamente en la memoria.

/memory permite acceder a los archivos de memoria (del usuario o del proyecto)

/export exporta la conversación completa a un archivo o al portapapeles

/btw (Claude code) sirve para platicar con el agente mientras trabaja (sin interrumpirlo)

Capacidades del Agente

- **Búsqueda web** — El agente puede buscar en internet para responder. Claude Code y OpenCode la traen integrada; en Codex hay que habilitarla en la configuración.
- **Pegar imágenes (multimodalidad)** — Puedes arrastrar una imagen a la terminal o pegarla con Ctrl+V y el agente la ve.
- **Procesos en segundo plano** — El agente puede lanzar un proceso que sigue corriendo sin bloquear la conversación, y seguir editando código; luego puede leer la salida de ese proceso para diagnosticar errores.
- **Lista de tareas** — En tareas largas, Claude Code muestra una lista de pendientes que el agente va marcando conforme avanza. Sirve para auditar en qué va y detectar si se desvió del plan.

Gracias

¿Preguntas?